

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesor:

Ing. Jorge Zambrano

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: Karen Elizabeth Velasquez Chaves

PARALELO: "A"

SEMESTRE: 5to semestres

ACTIVIDAD: # 1: Estudio de caso – Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2026 - ENERO 2027



Actividades a desarrollar en la Unidad # 1

Actividad # 1 Estudio de caso -sistema de control de rescate y adopción animal

1. Análisis del problema

1.1 Descripción del problema

El sistema de control de rescate y adopción animal tiene como finalidad gestionar los casos de animales rescatados que ingresan a un refugio. El sistema permitirá registrar los animales rescatados, clasificarlos según su especie, asignar un voluntario o veterinario responsable y realizar el seguimiento de cada caso mediante diferentes estados, como Rescatado, En Tratamiento y Adopción.

1.2 Actores principales

Ciudadano / Reportante:

Persona encargada de reportar un animal que necesita ser rescatado.

Voluntario / Veterinario:

Persona encargada de atender al animal, realizar el seguimiento del caso y actualizar su información y estado.

Administrador del Refugio:

Persona responsable de administrar los casos de rescate, asignar voluntarios o veterinarios y supervisar el proceso de atención y adopción.

1.3 Requerimientos funcionales

- **RF01:** Crear un caso de rescate.
- **RF02:** Registrar los datos del animal rescatado.
- **RF03:** Categorizar el animal según su especie: perro, gato o fauna silvestre.
- **RF04:** Asignar un voluntario o veterinario responsable.
- **RF05:** Actualizar el estado del caso.
- **RF06:** Consultar los casos de rescate registrados.
- **RF07:** Gestionar el proceso de adopción.
- **RF08:** Notificar a los voluntarios sobre nuevos rescates.

1.4 Requerimientos no funcionales

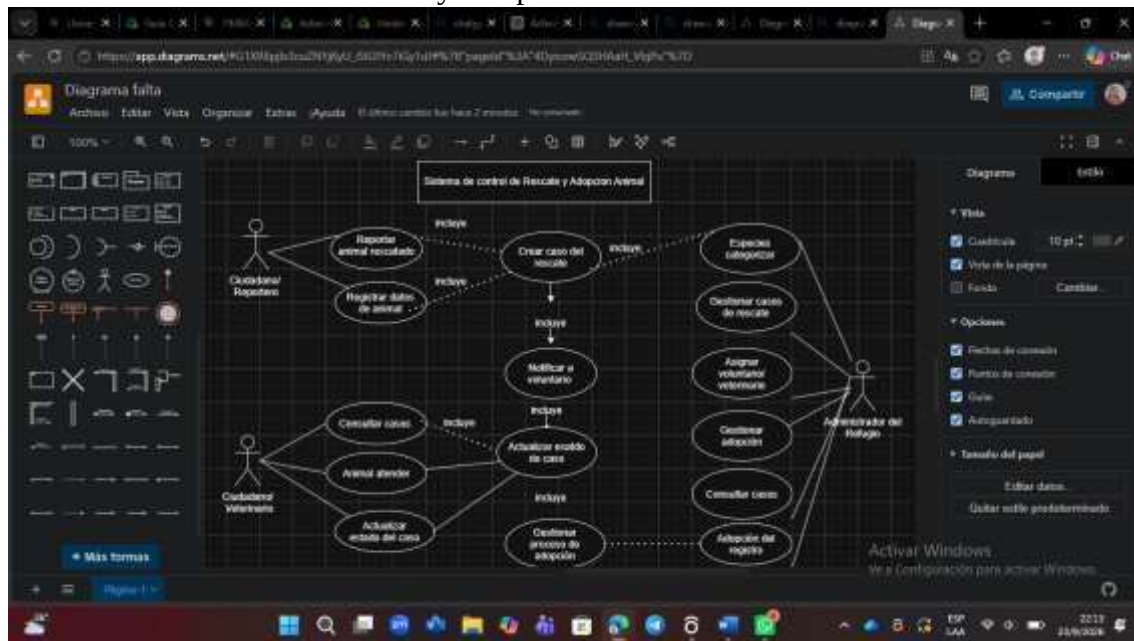
- **RNF01 – Escalabilidad:** El sistema debe permitir el crecimiento de la cantidad de casos y usuarios sin afectar su funcionamiento.
- **RNF02 – Mantenibilidad:** El sistema debe estar organizado de manera que facilite su mantenimiento y futuras modificaciones.

- **RNF03 – Trazabilidad de versiones:** Los cambios realizados en el proyecto deben poder ser identificados y controlados mediante Git y Gitflow.
- **RNF04 – Reutilización:** Los componentes del sistema deben estar diseñados para facilitar su reutilización.
- **RNF05 – Organización:** El sistema debe mantener una estructura clara que facilite su comprensión y desarrollo.

2. Diseño del sistema

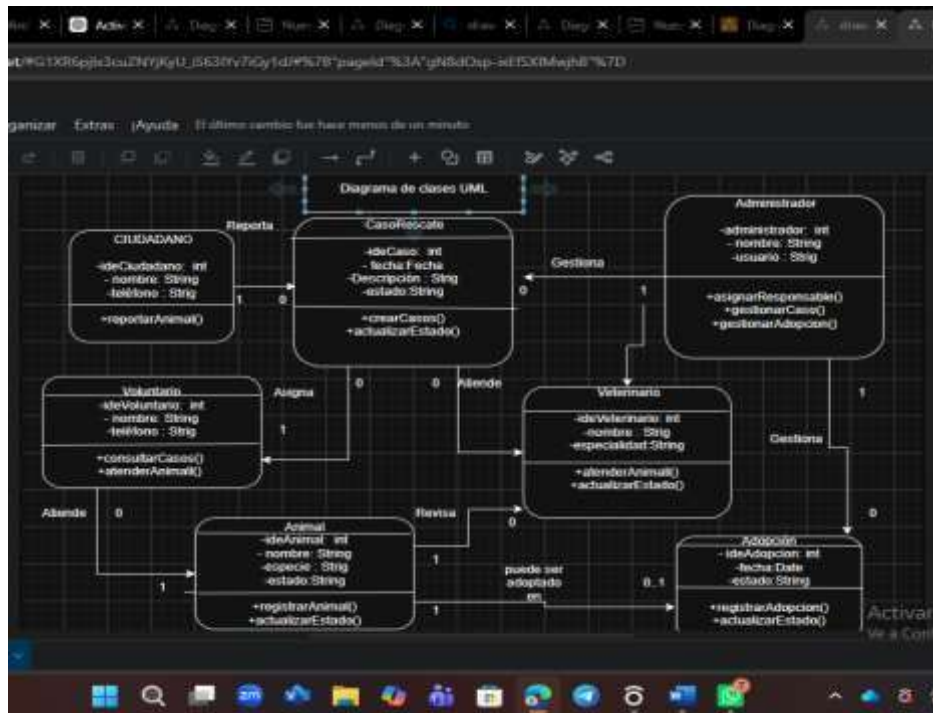
2.1 DIAGRAMA DE CASO DE USO

El diagrama de casos de uso representa las interacciones entre los actores principales y el Sistema de Control de Rescate y Adopción Animal.



2.2 Diagrama de clase UML

El siguiente diagrama representa las principales clases del Sistema de Control de Rescate y Adopción Animal, sus atributos, métodos y relaciones



2.3 Patrones de diseño

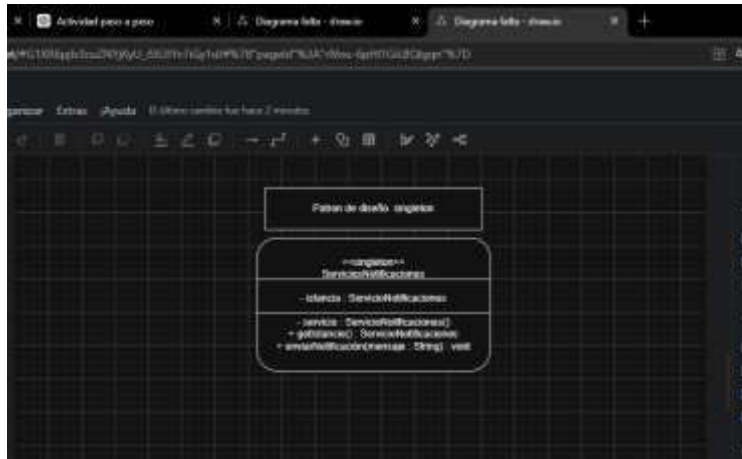
2.3.1 Patrón Singleton

Singleton

El patrón Singleton permite garantizar que una clase tenga una única instancia y proporciona un punto de acceso global a ella. En el sistema de rescate y adopción animal puede utilizarse para centralizar servicios que deben ser gestionados desde una única instancia, como un servicio de configuración o notificaciones.

Problema técnico que resuelve: evita la creación de múltiples instancias innecesarias de un mismo servicio y permite centralizar su acceso.

Aplicación en el sistema: se propone utilizar Singleton para un servicio centralizado de notificaciones, encargado de enviar avisos relacionados con los casos de rescate y sus cambios de estado.



2.3.2 Patrón Observer

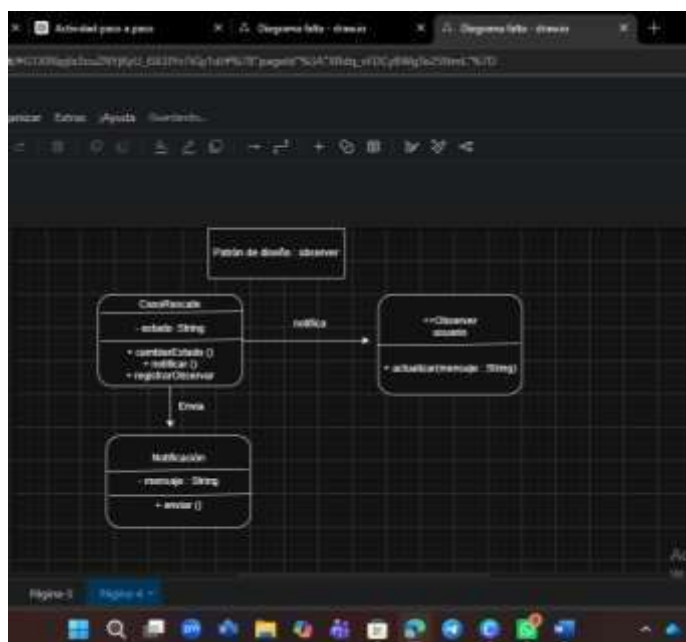
El patrón Observer permite que varios objetos sean notificados automáticamente cuando ocurre un cambio en otro objeto.

Problema técnico que resuelve:

Evita que el sistema tenga que consultar constantemente si ocurrió un cambio. Cuando cambia el estado de un caso de rescate, los usuarios interesados pueden recibir una notificación.

Aplicación en el sistema:

Se utilizará para notificar al ciudadano, voluntario, veterinario o administrador cuando cambie el estado de un caso de rescate, por ejemplo, de **Rescatado** a **En Tratamiento** o **Adopción**.



2.3.3 Patrón Factory Method

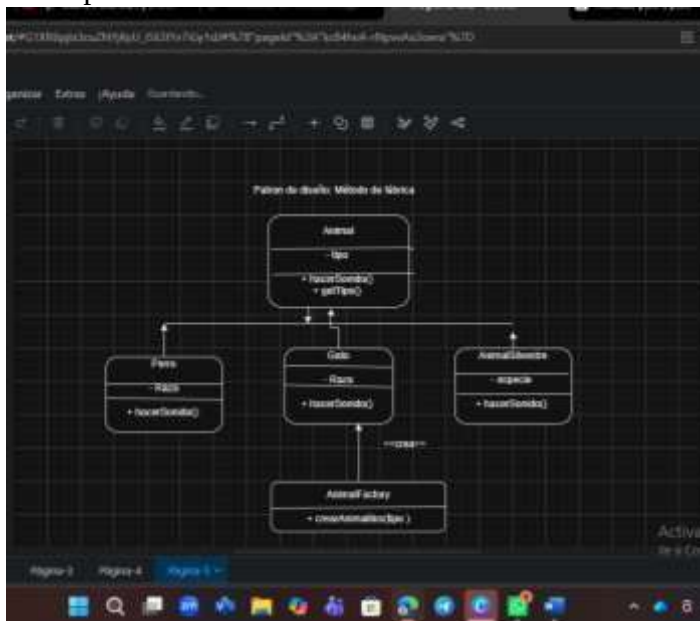
El patrón Factory Method permite crear objetos sin que el código principal tenga que conocer directamente la clase concreta que se va a instanciar.

Problema técnico que resuelve:

Evita que el sistema dependa directamente de muchas clases concretas al momento de crear objetos y facilita agregar nuevos tipos sin modificar demasiado el código existente.

Aplicación en el sistema:

Se utilizará para crear diferentes tipos de animales rescatados, como **Perro**, **Gato** o **AnimalSilvestre**, mediante una fábrica que se encarga de realizar la creación correspondiente.



2.4. FLUJO DE TRABAJO GITFLOW

2.4.1 Configuración del repositorio

Para el desarrollo del Sistema de Control de Rescate y Adopción Animal se creó un repositorio público en GitHub. El repositorio permite almacenar y controlar los cambios realizados durante el desarrollo del sistema. Para organizar el trabajo se implementó el modelo Gitflow mediante una rama principal, la rama `develop` y una rama `feature/casos-rescate`.

Repositorio: Sistema--recreate-adopcion-animal

Plataforma: GitHub

Visibilidad: Público

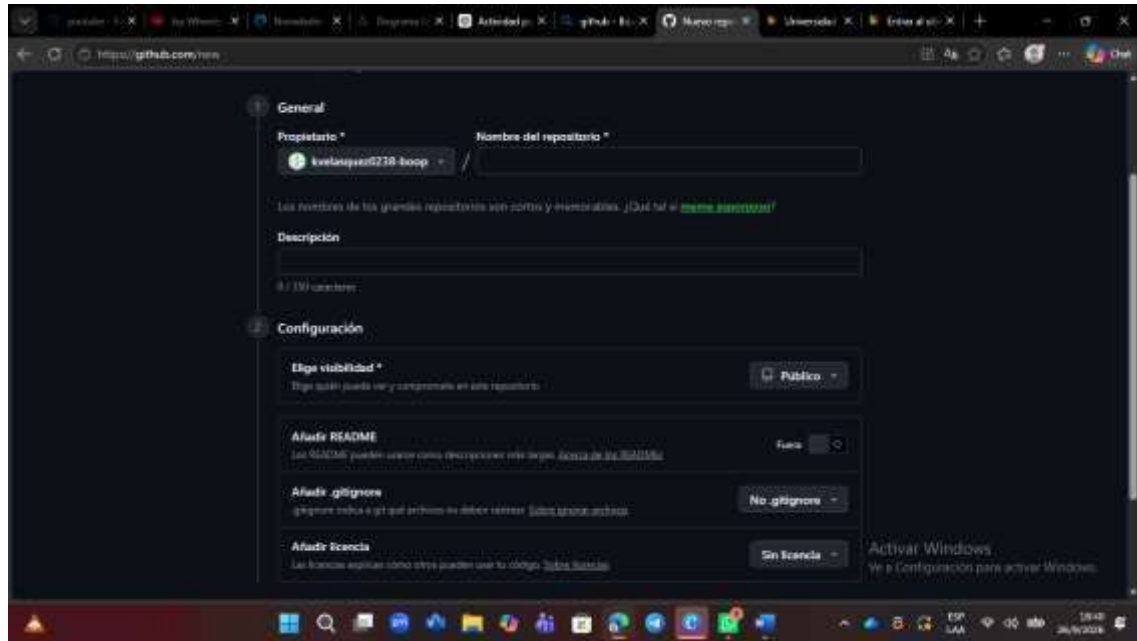
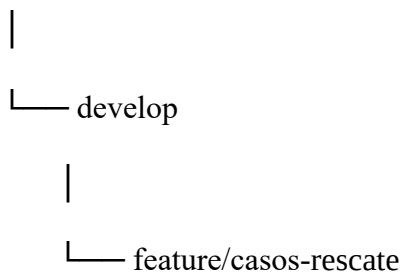


Figura: Repositorio público del Sistema de Control de Rescate y Adopción Animal en GitHub.

2.4.2 Estructura de ramas

Para organizar el desarrollo del sistema se utilizaron diferentes ramas siguiendo el modelo Gitflow. La rama `Principal` contiene la versión principal del proyecto, `develop` se utiliza para integrar los avances del desarrollo y `feature/casos-rescate` permite trabajar específicamente en la funcionalidad relacionada con los casos de rescate.

Principal



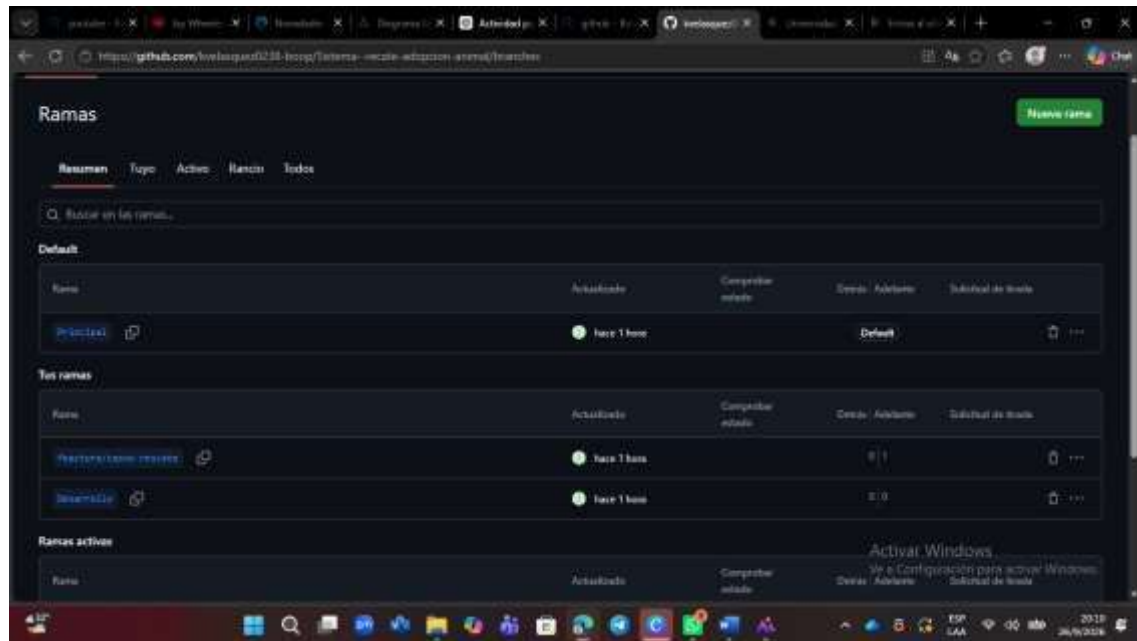


Figura: Estructura de ramas del repositorio GitHub. Se muestran la rama principal, la rama `develop` y la rama `feature/casos-rescate`, utilizadas para organizar el desarrollo del sistema mediante Gitflow.

2.4.3 JUSTIFICACIÓN DE GITFLOW

Se utilizó Gitflow para organizar el desarrollo del Sistema de Control de Rescate y Adopción Animal mediante diferentes ramas. La rama principal permite mantener la versión principal del proyecto, mientras que `develop` permite integrar los avances realizados durante el desarrollo. Las ramas `feature/` permiten trabajar de manera independiente en nuevas funcionalidades, como `feature/casos-rescate`.

El uso de esta estructura facilita la organización de los cambios, permite mantener un historial de commits y ayuda a realizar un seguimiento de las modificaciones realizadas durante el desarrollo del sistema.

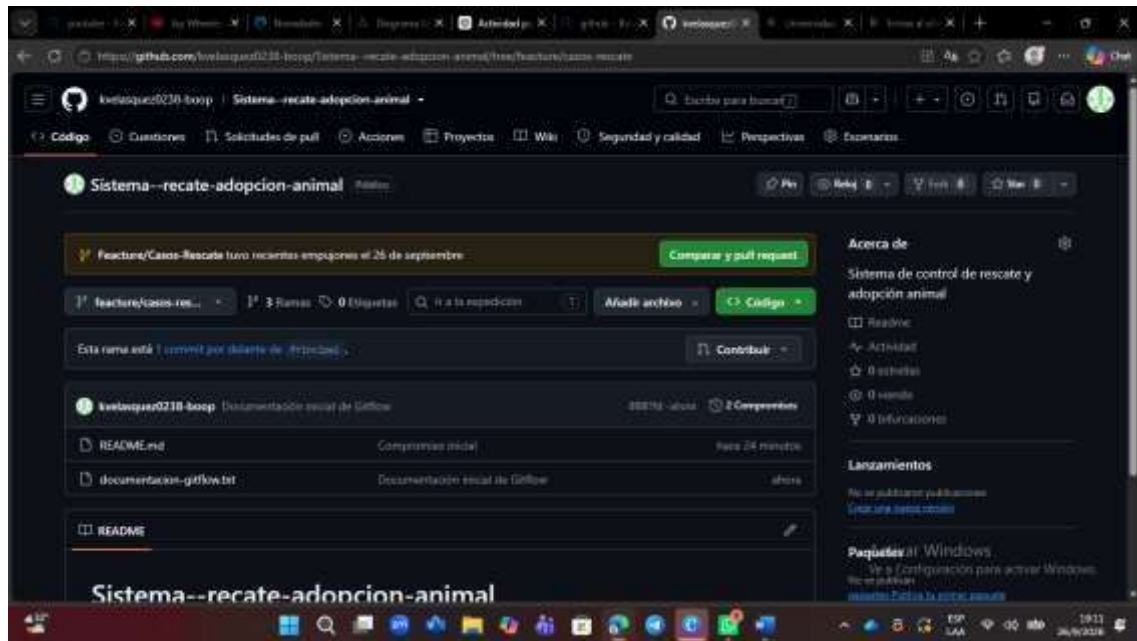
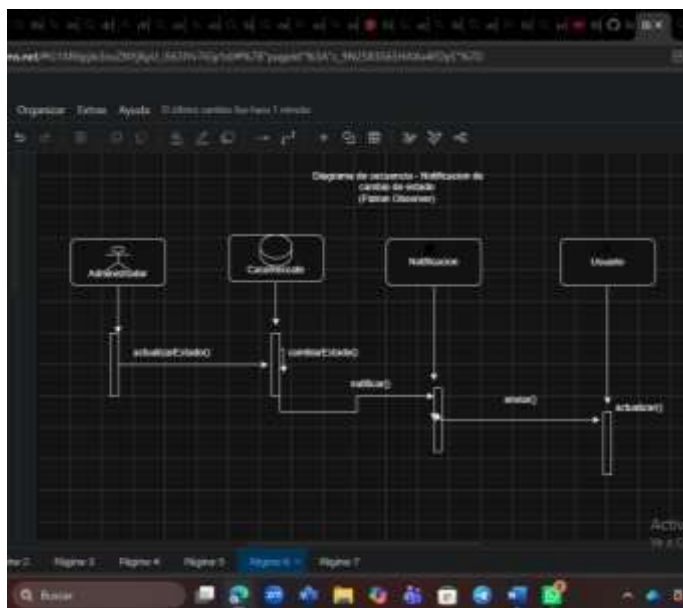


Figura: Historial de commits de la rama `feature/casos-rescate`.

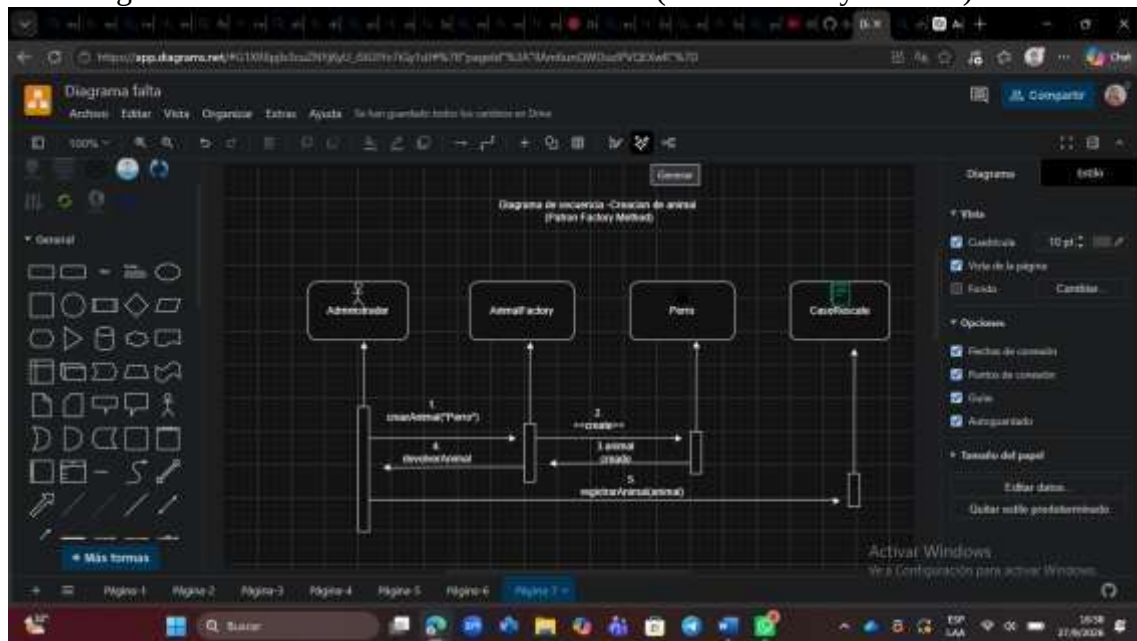
2.5 Diagrama de secuencia

2.5.1 Diagrama de secuencia – Notificación de cambio de estado



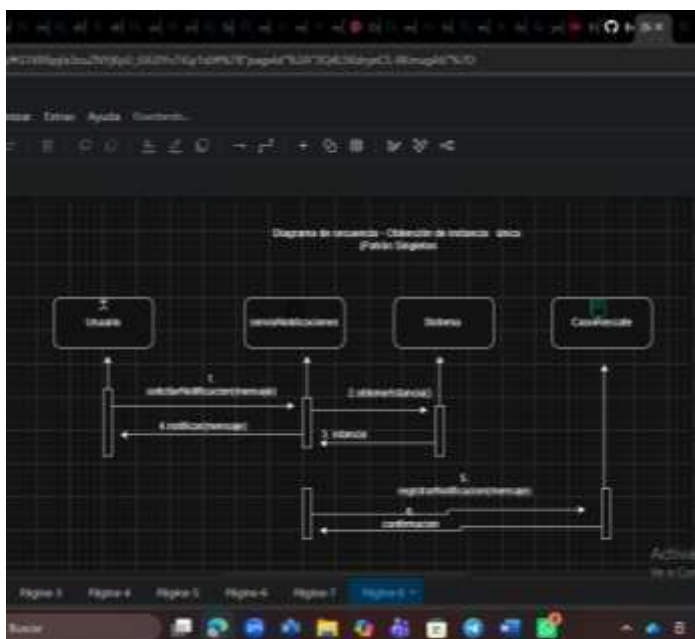
El diagrama muestra el proceso de actualización del estado de un caso de rescate. Cuando el administrador actualiza el estado, `CasoRescate` realiza el cambio y notifica a los usuarios mediante `Notificacion`. El usuario recibe el mensaje y actualiza la información correspondiente. Este flujo representa la aplicación del patrón Observer para gestionar las notificaciones automáticas

2.5.2 Diagrama de secuencia – Creación de animal (Patrón Factory Method)



El diagrama representa el proceso de creación de un animal mediante el patrón Factory Method. El administrador solicita la creación de un perro a AnimalFactory, que genera el objeto correspondiente y lo devuelve para que sea registrado en el caso de rescate. Este patrón permite crear diferentes tipos de animales sin que el administrador tenga que instanciarlos directamente

2.5.3 Diagrama de secuencia – Patrón Singleton



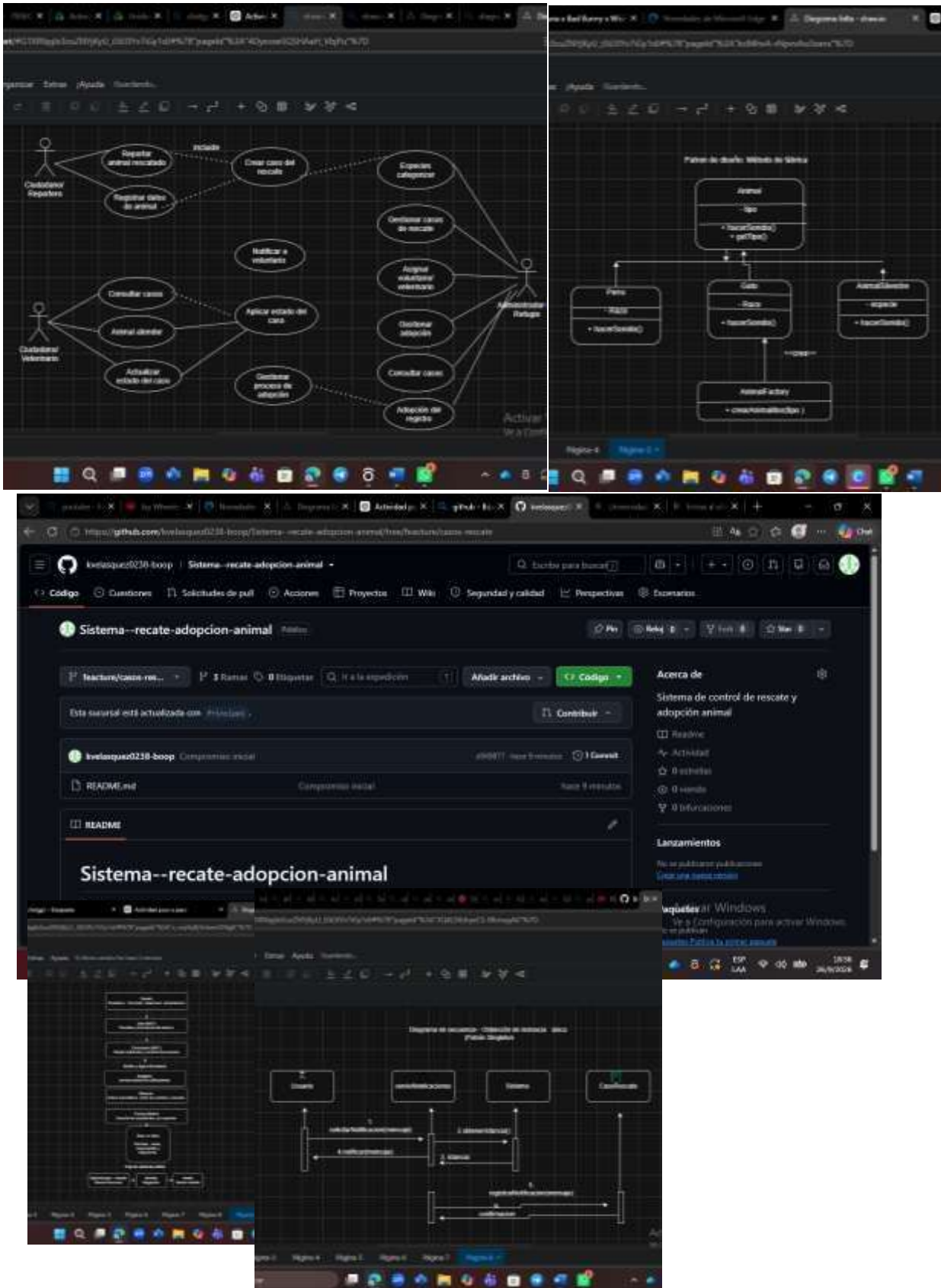
El diagrama representa el proceso mediante el cual un usuario solicita una notificación al sistema. Para gestionar el envío, se obtiene una instancia única del servicio de notificaciones, aplicando el patrón Singleton. Luego, el servicio envía el mensaje al caso de rescate y registra la notificación correspondiente. Este patrón permite centralizar el servicio y evitar la creación de múltiples instancias.

2.6 Arquitectura general del sistema



La arquitectura general del Sistema de Control de Rescate y Adopción Animal está organizada en capas que permiten una mejor distribución de las funciones. Los usuarios interactúan con el sistema mediante la vista, mientras que el controlador recibe las solicitudes y coordina las acciones con el modelo y la base de datos. Además, se incorporan los patrones de diseño Singleton, Observer y Factory Method para centralizar las notificaciones, informar sobre los cambios y facilitar la creación de registros de animales por especie. Gitflow permite organizar el desarrollo y el control de versiones del proyecto.

Evidencias





Se presentan las capturas de los diagramas UML, los patrones de diseño aplicados, la arquitectura general del sistema y el repositorio de GitHub. Estas evidencias permiten visualizar el proceso de diseño, la organización del proyecto y el control de versiones utilizado

ENLACES de Draw.io y Github

https://drive.google.com/file/d/1XR6pjlx3cuZNYjKyU_iS63lYv7iGy1dJ/view?usp=sharing Github

para visualizar los nueve diagramas abra el enlace y seleccione abrir con draw.io

<https://github.com/kvelasquez0238-boop/Sistema--recate-adopcionanimal/branches>

BIBLIOGRAFIAS

- GitHub. (s. f.). GitHub Docs.

<https://docs.github.com/>

- draw.io. (s. f.). UML diagrams.

<https://www.drawio.com/docs/diagram-types/uml/>

draw.io

- Object Management Group. (2017). Unified Modeling Language (UML), version 2.5.1.

<https://www.omg.org/spec/UML/2.5.1/About-UML>

OMG

- Refactoring.Guru. (s. f.). Design patterns.

<https://refactoring.guru/design-patterns/catalog>

Refactoring Guru

- draw.io. (s. f.). Drawing Gitflow diagrams in draw.io.



<https://www.drawio.com/docs/diagram-types/gitflow-diagram/>